

太原理工大学先进计算机系统实验室（ACSL）

适应期学员第三次学习路线

Brain Crisis!!

难度指数：024

报酬：专业程序员基础（2）

本次作业提交链接：[作业提交链接](#)

本周我们会学习到指针，结构体等比较困难的概念，大家在学习的过程中可能会遇到很多难以理解的问题，这时候请联系你的助教，他们会给你足够的帮助。不过提问前你需要先分清两种提问类型，一种是概念上的知识问题，另一种是代码上的实操问题。其中，代码的问题你需要给出 **你的完整源代码** 和 **较为具体的相关问题描述**。

陈浩男学长的碎碎念：本周我们已经要开始指针部分的学习了，大家会在本周遇到之前从来未体验过的困难，我在这里负责的告诉大家：“这是很正常的现象。”对于绝大多数的计算机专业同学来说，对指针的理解都是一件不简单的事情，比如我自己，第一次学习指针的时候，对于地址，内存，指向这些抽象的概念也是非常的困惑，直到很久以后了解了计算机更深入的知识（计算机组成原理），才对指针有了新的认识，而我们那个时候还没有组会这种引导形式。所以，如果在本周你遇到了困难，不要气馁，因为大家都不容易理解抽象的指针。借这次机会，我也鼓励大家多向自己的助教进行提问，以及与自己的同学积极讨论。如果在做了这些工作后，你仍然对指针有一点模糊，不要气馁，尽自己的所能完成任务，你的未知部分会在日后计算机知识体系完善起来之后慢慢揭开。

命令行基础

基本的命令

如果同学们对于Linux的命令行操作还不是很熟悉，可以通过以下2种途径来复习一下linux终端的基本操作。

- 网站：<https://missing-semester-cn.github.io/2020/course-shell/>（不用完成课后题，完成我们要求的题目即可）
- 视频：[常用的Linux命令介绍：13个基本命令和Shell脚本编程_哔哩哔哩_bilibili](#)

(视频后半部分内容涉及到shell脚本语法，大家暂时不用深究，学习使用终端基本命令即可，以及其中的where命令在大家的Shell上面是没有的，详情可以参考[这个文档](#))

然后完成以下练习：

！ 在做作业之前，请注意：

- 全部使用终端完成，不要使用图形界面
- 明确：什么是文件，什么是目录
- 遇到问题：STFW，以及提问群内的学长们

请根据如下的步骤完成作业：

1. 在家目录下创建目录 `file_operation`
2. 切换到该目录，使用 `pwd` 命令输出当前目录路径
3. 分别使用以下几种方式创建文件 `a.fasta`：
 - a. `echo + "内容" > 文件名` (>为初次，>>为追加文本内容)
 - b. `touch + 文件名 + nano编辑器`
4. `a.fasta` 内容如下：

代码块

```
1 Hello world!
2 My name is Bash!!
```

5. 使用 `file` 命令查看文件信息，使用 `cat` 命令、`less` 命令查看文件内容
6. 分别使用相对路径和绝对路径的方式列出 `file_operation` 目录下的文件以及大小
7. 将 `a.fasta` 拷贝为 `b.fasta`
8. 将 `a.fasta` 移动到 `/tmp` 目录
9. 将 `b.fasta` 文件压缩为 `b.fasta.tar.gz` (提示：使用 `tar` 命令，具体操作请参考[这个文档](#))
10. 删除 `b.fasta`
11. 解压 `b.fasta.tar.gz`
12. 删除 `b.fasta` 和 `file_operation` 目录
13. 使用 `history` 命令查看自己完成该练习的过程

14. 利用 `man` 命令阅读练习中使用过命令的详细说明，比如 `man ls`，`man cat` 等（了解有什么功能即可，需要用到时候查阅使用）。

命令行的小练习

完成后将你的 `.bash_history`（位于你的家目录下，隐藏文件在命令行下 `ls -a` 或者图形化界面 `ctrl + h`）复制到 姓名-专业班级-Great-3 文件夹中，等待作业提交。

C语言学习（其三）

从现在开始，我们就要进入C语言学习的难点了——指针和结构体。其中，指针是新手难以理解的一个概念，但也是C语言能够编写计算机底层代码、帮助我们学习计算机底层的最大优势。

不过首先要明白一点——指针像先前学过的 `int`，`char` 等一样，也是一种数据类型，存储的也同样是些数字，不要过度畏惧它。

更底层的世界

继续学习C语言，完成指针和结构体的学习。

接下来指针和结构体的学习，我们给出学习文档供大家参考：[📖 指针/结构体教程](#)。

练习

指针和结构体都在C语言中有着举足轻重的作用。前者是内存控制的重要工具，后者是数据结构的基石之一，往后的学习将离不开它们。

它们的练习题也有一定难度，根据上方的学习文档一步步完成[头歌湖南工业大学C语言](#)第7章的和第8章实验中的一部分练习。随后完成这个练习：[指针练习](#)。

（其中，如果你在某个题目的进度30min内无法有效继续推进，可以考虑暂时放弃并往后完成。）

头歌的题目我们不会直接提供答案，如果遇到了问题可以寻找群内助教，他们会给与一定的帮助。

完成后将**你完成的截图重命名为** `Thecoder2` 并放入 姓名-专业班级-Great-3 文件夹中，将你完成的 **指针练习的源代码文件** 按要求重命名后也放入该文件夹中。

 头歌的题目和学校里部分老师的课后题有很多重复，完成本次练习后，你的C语言课程基本可以高于85分，再学习一些书本理论知识这门课就差不多可以满足绩点了（运气不好，绩点也

跑)。至于其他老师，他们中大部分出题比这些还简单。所以往后学校的C语言课程对你们来说就是小菜一碟。

作业提交

作业提交

1. 基础作业 将你的 `.bash_history`，复制到 `姓名-专业班级-Great-3` 文件夹中
2. 基础作业 截图提交要求：将你完成**头歌指定题目的截图**重命名为 `Thecoder2` 并放入上述文件夹中
3. 基础作业 源代码提交要求：将你的源代码文件也放在上述文件夹中。命名见练习题目要求。
4. 将文件夹压缩为zip格式，保持压缩包名字与文件夹名字相同并提交至到 [第三周作业提交表单](#)
5. 如果你学有余力完成了下面的拔高**必做**内容，则把文件夹重命名，格式为 `姓名-专业班级-Newstar-3`。不要忘记更新你的源代码和截图，还有提交表单。

本周作业提交截止时间：**11月8号晚23:00**（如果因为自身原因未完成本周作业，在表单**请假说明**一栏填写原因，并写上你的预计提交时间即可）

必做：[链接1](#) [链接2](#) [链接3](#)

拔高：[链接1](#) [链接2](#) [链接3](#)

拔高内容

折腾你的 Shell

这一次的拔高没有Shell的编程任务，这次我们只需要去解决这样有一个问题——如何用Shell改变Shell！

你不需要完全理解这些他们的原理，需要的是通过动手体验，知道 `原来有这样的工具` / `有这样的操作`，在以后真正需要时就能轻松查资料解决问题。

通过前期学习，大家已经大体了解了什么是 Shell 并掌握了 Shell 的基础操作。从本周开始，我们将一起解锁 Shell 的进阶技巧，让你的命令行体验更高效、更酷炫！

概念区分

当你在电脑上打开终端时都发生了什么？这里我们仅需了解其中两个最重要的两个组件：

- 终端：让你的操作可视化的窗口——可以理解为电视屏幕
- Shell：同终端交互的工具，解析和解释你的操作的翻译官——就像电视的遥控器

可见，前面我们所实现的简易终端实质上指的是简单的Shell，但由于Shell的概念传播不如终端这个词广泛，我们便在前期使用终端一词。

终端的配置大多可以用图形化的界面来更改，大家在前两周或许就已经有所接触（比如让你的终端窗口透明化，字体更加美观等）。

但我们该如何修改Shell的配置呢？我们以应用最广泛的一种Shell——Bash为例。

Bash

Shell的种类有很多（Bash, csh, dash, zsh...），而Ubuntu自带的Shell就是应用最广泛的Bash

当你想个性化某款软件的功能和外观时，你需要找到那个软件的“设置”，然后更改其中的选项“告诉”软件你想要的效果，从而实现个性化。

但是对于没有图形化界面的Shell，我们想要更改它就只能通过Shell本身的性质来进行更改——比如一直使用的各种命令。

但Shell被启动前是无法被我们用输入命令这种方式来进行更改的，这时候我们就需要一个脱离Shell/终端体系的一种东西——文件。文件独立于它们存在，也可以在它们未启动时进行更改。

这时就需要自己去写“配置文件”来“告诉”Shell：我想要什么样的效果。这个配置文件就是Shell配置。

通过前面的学习，你已经知道Shell是命令行的“解释器”，负责接收并执行你的指令。其实它也像一个小软件，可以被你“定制”出独一无二的样子。

? 文件可以存储我们需要的更改，具体地就是用文字存储一些用于更改的命令/操作，这样的文件我们称之为配置文件。

尽管这种配置方案比较适合像Shell这种没有图形化界面的软件，但现在部分具有图形化的软件（比如游戏）仍然保留了用户可编辑的配置文件来进行较为隐蔽的配置更改。

当你打开终端时，Shell会自动读取一个隐藏文件：`~/.bashrc`

在文件名前方有 `.` 符号的文件将在ls命令下隐藏，因此被称为隐藏文件。为了观察到他们，我们需要使用 `ls -a` 或 `ls -al` 命令：

代码块

```
1 $ ls -al
2 [.....]
3 -rw-r--r-- 1 hoan hoan 5430 Oct 15 22:32 .bashrc
4 [.....]
5
```

其中里面具体的输出每个人的都是不同的，但文件名 `.bashrc` 都是一样的。

那么 `.bashrc` 里都有什么呢，又都能写什么呢？

就像前面所说，里面存储了一些命令/操作，在这里存储的就是命令。当Shell被启动时它将会逐行读入并执行其中存储的命令，这样就做到了当我们使用时配置生效的效果。

这样就能够解释 `bashrc` 中的 `rc` 是什么意思，其实就是 `run command`。

！ 为什么需要重启你的计算机

读到这里，你就或许意识到了为什么经常有软件在更改后需要你“重启计算机以应用新的更改”，原来计算机也有像这样类似的操作！这也许就能够作为“重启解决99%的问题”这句名言的一个解释。

备份

为了避免一些不必要的问题，我们要求你在执行后面的操作之前执行这样的命令：

```
mkdir -p ~/backup; cp ~/.bashrc ~/backup;
```

动手试一试

执行 `echo "echo Hello, World!" >> ~/.bashrc` 命令，然后打开一个新的终端，这时你会看到什么呢？

能不能不打开新的终端来应用你的更改呢？当然是可以的，我们需要 `source` 命令，只需要输入：

代码块

```
1 source ~/.bashrc
```

这条命令的意思是“重新加载这个配置文件”，就像刷新网页一样。

现在，让它在启动时打出你自己的名字。执行：

```
代码块 echo 'echo Welcome back, My name is xiaoming! ' >> ~/.bashrc
```

你也可以在里面加上时间：

代码块

```
1 echo 'echo today is: $(date)' >> ~/.bashrc
2 # 执行时$(date)将会被替换为当前日期
```

你也可以试着把里面的 `date` 替换为 `date +%Y-%m-%d %H:%M:%S`

完成后你的终端在启动后就会向你发送欢迎词以及当前时间。

设置命令别名和环境变量 —— 让终端更“便捷”

别名——alias

每次输入一个经常使用但很长的命令时我们都会头疼能不能有一种工具来简化这种操作。Shell的确有这样的一个命令可以做到这样的效果——`alias`。

我们可以执行以下的命令：

代码块

```
1 $ say Hi
2 $ alias say='echo'
3 $ say Hi
```

这样我们就可以用say来执行 `echo` 的命令了，后面的 `say Hi` 就等价于 `echo Hi`，这就帮助我们节省了一个字母的时间。

但你会发现这个终端退出后你就无法继续使用 `say` 了，这说明我们需要在每次Shell启动后运行 `alias say='echo'` 才行。而我们刚好可以用`.bashrc`来做到这样的效果。

例如，你可以执行以下命令来更改你的 `.bashrc`：

代码块

```
1 $ echo "alias update='sudo apt update && sudo apt upgrade'" >> ~/.bashrc
2 $ source ~/.bashrc
```

之后你只需要在终端输入 `update`，它就会自动执行整条更新命令，等价于执行 `sudo apt update && sudo apt upgrade`。是不是像给终端加了一个“快捷指令”？

环境变量——export

除了给命令起“外号”，我们还可以通过“环境变量”让终端更“懂你”。

但什么是环境变量呢？和普通变量有什么不同？简单说，环境变量就像系统或程序的“小纸条”，上面记着一些常用信息（比如路径、名称、配置等），方便 Shell 或其他程序随时“查看”。

比如你可能听过 `PATH` 这个环境变量，它就像一张“地图”，告诉系统“去哪里找你输入的命令”——当你在终端输入 `ls` 时，系统就是通过 `PATH` 找到 `ls` 命令的位置并执行的。

环境变量由 `变量名=值` 的语法组成，比如 `USER="your name"`，其中 `USER` 是变量名，“your name”是变量值。在 Shell 中，用 `$变量名` 可以读取它的值，比如执行 `echo $USER`，终端就会打印出“your name”。

我们可以在 `.bashrc` 中使用 `export` 命令自定义环境变量，例如 `export USER="your name"`，让终端“记住”你常用的信息，比如你的名字、常用文件夹的路径等。



加功能，写注释

在你的 `.bashrc` 最后新增一些功能，比如设置你的桌面路径为 `DESKTOP` 环境变量，或者把你的名字设置成 `MY_NAME` 环境变量。在你的更改后面用 `# xxx` 写上功能注释。

完成后将你的 `.bashrc` 复制到 姓名-专业班级-NewStar-3 文件夹中，等待作业提交。

? 思考一下

1. 为什么系统每次打开终端时，都会执行 `.bashrc` 里的内容？
2. 如果 `.bashrc` 里什么都没有，bash 还能正常用吗？
3. 你还能想到哪些命令可以写进去，让终端更方便或更好玩？

source命令

前面我们使用了 `source` 来更新Shell的配置，那么为什么 `source` 命令可以做到这个呢？



RTFM

阅读 `help -m source` 的 `DESCRIPTION` 内容，了解 `source` 命令的工作原理，并思考如何将这个命令运用到其他Shell脚本当中去？



点命令 (.)

阅读 `help -m .` 并与 `help -m source` 进行比较，`.` 命令与 `source` 是否有不同？

指针拔高内容

二级指针

！ 二级指针

接下来要提到的类型为 `void**` 的东西其实是个二级指针类型。但是void类的指针都不是那么好理解，我们先用int类的指针作为引导。

什么是二级指针呢？其实就是指向一个指针的指针。我们先从一级指针开始，如下：

代码块

```
1  int array[10];
```

这是一个一维数组，`array` 的类型虽然不是指针，但我们在把它作为赋值语句的**右侧**时可以把它当做指针来使用，此时可当做指向数组首个元素的指针，这是编译器允许的行为。

所以一个指向array第一个元素的指针可以这么写：

代码块

```
1  int array[10];  
2  int *p = array;
```

我们刚刚定义的p就是一个**一级指针**了，也就是大家学的普通的指针。

而**二级指针**是指向**一级指针变量**的指针，因此应该这样写：

代码块

```
1  int array[10];  
2  int *p = array;  
3  int **dp = &p;
```

因为是二级指针，所以要用两个*号指示。又因为是指向**指针变量**，所以要用取址符 `&` 把 `p` 的地址传入 `dp`。

`dp` 作为二级指针可以进行两次解引用，第一次解引用表示其指向的一级指针 `p`，二次解引用为 `p` 的解引用结果，也就是 `array` 的第一个元素。而二级指针可以做到更换其指向的一级指针，就像如下代码：

代码块

```
1  #include <stdio.h>
2  int main(){
3      int array1[10] = {9,};
4      int *p = array1;
5
6      int array2[10] = {10};
7      int *q = array2;
8
9      int **dp = &p;
10     printf("%d\n", **dp);
11     dp = &q;
12     printf("%d\n", **dp);
13 }
```

这做到了在两个数组间进行访问切换。尽管看起来用处不大，但你应该能看出它的潜力——二级指针相对一级指针拥有更大的内存访问区域。理解这个对你以后学习 `计算机组成原理——内存篇` 有极大的帮助。

array的类型到底是什么？

[这里](#)提到的 `array` 其实并不是指针类型。不过有许多教程都把 `array`（数组名）这样的东西当做普通指针（或指针常量，即指针本身是个不可改变其指向的常量）来解释，却不管它们实质上的不同。

言归正传，现在需要你 **STFW** 和 **RTFM**，去了解到底什么是 `array`（数组名）。但是这个 **STFW** 难度很大，因此我给它标了红。前面也说过有许多教程给出的是错误的内容，你很难保证 **STFW** 后自己获得的“知识”是正确的。不过 **RTFM** 难度也不小，所以这个任务不要求你提交作业。

此处不建议通过编译器（语法分析）是否报错来进行判断，因为其报错的原理有时并不是完全按照标准进行的，有些难以实现的 `向你报告这里有一个错误` 的功能有可能并没有实现。

在找到答案后，尝试用 `sizeof()` 来判断 `arr` 的大小和指向 `arr` 首元素 `指针` 的大小，并于运行前做出预测，以此验证你的答案。

指针函数——自制 `strcmp`

既然函数有返回值，参数也可以传入指针，那么我们能不能让函数返回值是一个指针呢？显然是可以的，返回值类型完全由我们程序员决定。

你需要实现一个 自制的 `strcmp`，和库函数（[什么是C库函数](#)）的 `strcmp` 不同，它需要返回的不是数，而是指向某个字符串的二级指针。要求如下：

1. 定义一个 `my_strcmp` 函数，它的返回值应是 `char **`
2. 它能够接收两个参数，它们的类型均为 `char *`
3. 对传入的两个字符串进行比较，并返回指向更小的字符串的二级指针
4. 若相等则返回一个 空指针

注：你可以使用库函数的 `strcmp` 来进行比较，也可以自己实现其功能。

完成后将你的源代码重命名为 `new_strcmp` 并放入 姓名-专业班级-Great-3 文件夹中，等待作业提交。

优化

我们实现的这个 自制 `strcmp` 函数实际上是非常不安全也不是很好用，所以我们给你如下改进方向：

1. 添加一个要传入的参数，类型为 `int`，能够控制需要比较的字符数
 - a. 即比较过两个字符串的前 `n` 个 字符后仍相等则返回一个 空指针
2. 添加一个传入的参数，类型为 `int`。其值非0时返回比较结果为指向较大字符串的二级指针。

完成后将你的源代码重命名为 `new_strcmp2` 并放入 姓名-专业班级-Great-3 文件夹中，等待作业提交。

? 为什么要返回二级指针？

当我们能够返回二级指针，那么我们就可以用上前面指针数组的知识来储存返回的字符串，这样我们就能制作一个“字符串数组”。不过其实这里的主要目的是练习你的函数、指针和数组的综合能力，实际运用时大多会使用返回一级指针（指向字符串首元素）的方案。

温馨提示

第三次学习路线到此结束，加油

本作品《"太理先进计算机系统研究实验室前置讲义适应期培养篇"》由 张勇俊 创作，并采用 CC BY-SA 4.0 协议进行授权。

遵循CC BY-SA 4.0开源协议：<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.en>

转载或使用请标注所有者：张勇俊，太理先进计算机系统研究实验室